Solution Architecture

Phase 3 | Application components and interfaces

Component view

```
Browser
React application (App.jsx, pages, components)
| frontend/src/services/api.js
| HTTPS JSON + bearer access token
v
FastAPI app (main.py)
| /auth -> auth service -> Supabase or memory
| /trips -> trip/itinerary services -> Supabase or memory
| -> AI orchestrator -> Groq / Ollama / local generator
`-> history service -> Supabase or memory
|
`-> Pydantic schemas define request/response shapes
```

Interfaces

Interface	Contract
Frontend to API	JSON over HTTP; Authorization bearer token on protected endpoints.
API to AI provider	Provider abstraction accepts system and user prompts; response parsed as structured itinerary JSON.
API to database	Backend service-role Supabase client; application layer scopes trip operations by authenticated user.
Frontend to browser storage	Access token/profile identifiers and selected saved-trip cache values in localStorage.

Deployment view

The repository includes a backend Dockerfile, frontend Dockerfile and Nginx configuration, docker-compose.yml, a Render service definition, and Vercel route rewrite configuration. Production values are supplied through environment variables. The actual production deployment state must be verified independently.

Architecture considerations

- Do not expose Supabase service-role credentials to the browser.
- Maintain CORS origins for the actual deployed frontend domain.
- Treat in-memory storage as a development convenience only.
- Reconcile the duplicate schema file under server/src/config if both copies are expected to stay authoritative.
- Document that Vercel trip routes require authentication unless a separate public-sharing design is added.

Prepared from the current project source and configuration. Product assumptions and proposed work are labeled; no user research, test result, or deployment result is implied unless explicitly stated.